

Trabajo Práctico: Tax-as-Code

Materia: 72.39 Autómatas, Teoría de Lenguajes y Compiladores

Equipo G-124

Nombre	Apellido	Legajo	E-mail
Santiago	Cibeira	64560	scibeira@itba.edu.ar
Manuel	Suñol	64138	msunol@itba.edu.ar
Augusto	Ospal	63669	aospalmadeo@itba.edu.ar

Tabla de Contenidos

1. Introducción
2. Modelo Computacional
 - 2.1. Dominio
 - 2.2. Lenguaje
3. Implementación
 - 3.1. Frontend
 - 3.2. Backend
 - 3.3. Adicionales
 - 3.4. Dificultades Encontradas
4. Futuras Extensiones
5. Conclusiones
6. Referencias
7. Bibliografía

1. Introducción

El presente proyecto tiene como objetivo el diseño e implementación de **Tax-as-Code (TaC)**, un Lenguaje de Dominio Específico (DSL) enfocado en la declaración de normativas fiscales, reglas de facturación y retenciones impositivas para el ecosistema corporativo argentino.

La principal problemática que aborda el proyecto es la fragilidad de los sistemas contables actuales (ERPs genéricos), donde la configuración de topes impositivos y alícuotas se realiza de forma manual en bases de datos o requiere la intervención directa del área de Sistemas ante cada cambio regulatorio. **Tax-as-Code** permite a los profesionales contables "legislar" las reglas mediante archivos de texto plano (`.tac`) con una sintaxis natural y amigable, generando una fuente de verdad única, versionable (mediante Git) e inmutable.

El compilador fue construido siguiendo el pipeline clásico de compilación descrito por Aho et al. (2006), utilizando las herramientas **Flex** y **Bison** sobre lenguaje **C**, tal como se describe en Levine (2009).

2. Modelo Computacional

2.1. Dominio

El dominio seleccionado abarca la configuración, automatización y aplicación de reglas de facturación, retenciones impositivas (Ganancias e Ingresos Brutos), pagos a proveedores y controles fiscales dentro del ecosistema corporativo argentino. La necesidad surge de la fricción operativa que sufren las empresas al lidiar con regímenes tributarios cambiantes, padrones mensuales de ARBA/AGIP y auditorías de riesgo fiscal.

El paradigma adoptado es **declarativo**: el usuario describe *qué* reglas fiscales deben aplicarse y bajo *qué* condiciones, sin especificar *cómo* se evalúan internamente. Esto oculta la complejidad algorítmica al profesional contable y favorece su experiencia de uso (DX).

2.2. Lenguaje

El lenguaje TaC ofrece las siguientes construcciones principales:

- **Constantes (definir)**: Permiten declarar valores reutilizables con soporte para expresiones aritméticas y porcentajes. Ejemplo: `definir TOPE_RETENCION: 1000000` , `definir TASA: 6%` , `definir MINIMO: 500000 + 200000` .
- **Reglas (regla_pago , regla_facturacion , regla_exencion)**: Bloques que encapsulan un conjunto de condiciones bajo un nombre descriptivo, agrupados con llaves `{ }` .
- **Condiciones**: Expresiones relacionales con operadores en lenguaje natural (`igual a` , `mayor o igual a` , `menor o igual a`). Los valores pueden ser números, cadenas de texto, booleanos (`verdadero / falso`) o referencias a constantes previamente definidas.
- **Expresiones aritméticas**: Soporte para operaciones `+` , `-` , `*` , `/` tanto en definiciones de constantes como en condiciones de reglas (ej: `monto menor o igual a TOPE * 2.5`).
- **Listas**: Mediante corchetes `[ ]` para agrupar múltiples valores.
- **Comentarios**: De línea simple utilizando `//` .

Ejemplo representativo del lenguaje:

```

definir TOPE_RETENCION: 1000000
definir FACTURACION_MINIMA: 500000 + 200000
definir SCORE_CRITICO: 50 * 2
definir TASA_GENERAL: 6%

regla_pago "Retencion Ganancias – Alquiler" {
    - es_monotributista igual a falso
    - concepto igual a "alquileres"
    - monto mayor o igual a TOPE_RETENCION
}

regla_facturacion "Alerta Control Fiscal" {
    - es_monotributista igual a verdadero
    - monto mayor o igual a FACTURACION_MINIMA
    - score mayor o igual a SCORE_CRITICO
}

regla_exencion "Exencion PyME" {
    - es_pyme igual a verdadero
    - monto menor o igual a TOPE_RETENCION * 2.5
}

```

3. Implementación

3.1. Frontend

Análisis Léxico (Flex)

Se definieron patrones regulares (`FlexPatterns.l`) para reconocer los tokens del lenguaje. Se abstrajeron conceptos numéricos (`NUMBER`), cadenas de texto (`STRING`), identificadores de variables (`IDENTIFIER`) y palabras clave reservadas (`KW_REGLA_PAGO` , `KW_IGUAL_A` , `KW_MAYOR_O_IGUAL_A` , etc.). El lexer filtra automáticamente los espacios en blanco, tabulaciones y comentarios de línea (`//`).

Análisis Sintáctico (Bison)

Se estableció una gramática libre de contexto de tipo LALR(1) (`BisonGrammar.y`) que define la estructura jerárquica del lenguaje. La gramática soporta:

- **Declaraciones:** Definición de constantes con expresiones aritméticas (`definir TOPE: 1000 + 500`).
- **Reglas:** Estructuras que encapsulan un conjunto de condiciones (`regla_pago "Nombre" { ... }`).

- **Condiciones:** Expresiones relacionales (monto mayor a 1000) con soporte para listas de strings, booleanos y referencias a constantes.

Durante esta fase se resolvieron conflictos de tipo *Shift/Reduce* mediante directivas de precedencia para diferenciar operadores aritméticos de viñetas de lista.

Árbol Sintáctico Abstracto (AST)

El parser de Bison construye en memoria un Árbol de Sintaxis Abstracta (AST) definido en `AbstractSyntaxTree.c`. Cada nodo representa una abstracción semántica (Programa → Declaraciones → Regla/Constante → Condiciones → Valores), permitiendo desacoplar completamente la fase de análisis de la fase de generación de código.

3.2. Backend

Análisis Semántico

Un módulo especializado (`SemanticAnalyzer.c`) recorre el AST verificando la coherencia del programa antes de la generación de código. Entre sus validaciones:

- **Tabla de Símbolos (`SymbolTable.c`):** Registra las constantes declaradas mediante `definir` y verifica que toda referencia posterior a un identificador corresponda a una constante previamente definida. Detecta redeclaraciones duplicadas y variables no declaradas.
- **Chequeo de Tipos:** Asegura que no se comparen variables booleanas con cadenas de texto o números, previniendo errores semánticos en tiempo de compilación.

Generación de Código

En línea con las especificaciones del Stage III, el compilador (`Generator.c`) exporta la lógica de negocio a los siguientes artefactos de salida:

- **Archivos Planos (.txt) Normativos:** Archivo `afip_arba_configuration.txt` exportado en formato posicional de longitud fija, compatible con la estructura de importación de aplicativos fiscales (SICORE, SIAP).
- **Planilla de Auditoría (.csv):** Archivo `audit_matrix.csv` con formato tabular que desglosa cada regla con sus condiciones, operadores y valores esperados, facilitando la conciliación y auditoría por parte de los contadores.
- **JSON Estructurado:** Un archivo `tax_rules.json` que representa matemáticamente las reglas y condiciones en un formato estándar, listo para ser consumido por cualquier sistema externo (véase Sección 3.3).

3.3. Adicionales

Motor Transaccional: Mini ERP en Golang

Para demostrar la versatilidad del archivo `tax_rules.json` generado por el compilador, se construyó un sistema satélite: un **Mini ERP desarrollado en Go** con interfaz web.

Este sistema está completamente **contenedorizado utilizando Docker** y corre de forma independiente al compilador. Se integra en el mismo archivo `compose.yaml` bajo el servicio `mini-erp`, exponiendo una interfaz web en el puerto `8080`. Su función es recibir facturas entrantes, cargar en vivo el archivo `tax_rules.json` (gracias a un volumen compartido de Docker) y evaluar las condiciones fiscales.

El sistema liquida impuestos bajo dos esquemas independientes y complementarios:

1. **Retención de Ganancias:** Evaluación dinámica de las condiciones declaradas en el archivo `.tac` compilado (eximiendo automáticamente a monotributistas y aplicando alícuotas del 6.00% sobre conceptos como alquileres).
2. **Retención de Ingresos Brutos (IIBB):** Consulta a un padrón tributario simulado, implementado como un mapa de CUITs hardcodeados en el código Go. Cada entrada del padrón almacena la condición fiscal, la alícuota y la jurisdicción correspondiente al CUIT (ARBA para Provincia de Buenos Aires, AGIP para CABA). Si un CUIT no se encuentra en el padrón, el sistema bloquea la operación e informa que no es posible liquidar impuestos para contribuyentes no empadronados. En un entorno productivo, este mapa sería reemplazado por consultas a los Web Services reales de ARBA y AGIP.

Al procesar una factura de un contribuyente empadronado, el sistema calcula ambas retenciones de forma diferenciada, liquida el importe neto a pagar y emite certificados oficiales separados para cada tributo.

3.4. Dificultades Encontradas

- **Conflictos Shift/Reduce en Bison:** La convivencia entre operadores aritméticos (`+`, `-`, `*`, `/`) dentro de expresiones de constantes y el uso de `-` como viñeta de lista para las condiciones de las reglas generó conflictos de ambigüedad que requirieron un manejo cuidadoso de las directivas `%left` y `%right` de precedencia en la gramática.
- **Gestión de memoria del AST:** La construcción dinámica de nodos del AST con `malloc` en C demandó una implementación disciplinada de la liberación de memoria, evitando *memory leaks* en programas de entrada extensos.
- **Integración Docker entre compilador y Mini ERP:** Lograr que el servicio Go leyera en caliente el archivo `tax_rules.json` generado por el compilador en C requirió diseñar correctamente los volúmenes compartidos en `compose.yaml` y la estructura de directorios de trabajo dentro de cada contenedor.

4. Futuras Extensiones

- **Conexión a APIs reales de AFIP/ARBA/AGIP:** Reemplazar el padrón tributario simulado por consultas en tiempo real a los Web Services de ARCA (ex-AFIP), ARBA y AGIP, utilizando autenticación por certificados digitales.
- **Soporte para entidades (cliente , proveedor):** Extender la gramática para permitir la instanciación de actores comerciales con atributos fiscales (tipo de persona, condición IVA, jurisdicción), tal como se propuso en la especificación original del Stage I.
- **Tareas asincrónicas (sincronizacion_padrones , cierre_mensual):** Implementar bloques de automatización para programar la descarga periódica de padrones y la generación automática de reportes de liquidación.
- **Generación de código a lenguajes de propósito general:** Emitir código Python o C que actúe como motor de reglas ejecutable de forma autónoma, sin dependencia del Mini ERP.
- **Soporte multi-jurisdicción simultáneo:** Permitir que un mismo CUIT tenga alícuotas diferenciadas en múltiples provincias, evaluando Convenio Multilateral.

5. Conclusiones

La separación de responsabilidades demostró ser altamente efectiva. Mediante **Tax-as-Code**, la lógica impositiva volátil queda en manos del sector contable bajo estricto control de versiones (Git), mientras que los sistemas de gestión (como el Mini ERP) se limitan a leer un JSON estándar, garantizando una arquitectura robusta, auditable y libre de fallas críticas en los despliegues.

El compilador cumple con las cuatro fases fundamentales del pipeline de compilación (análisis léxico, sintáctico, semántico y generación de código), y la integración con un sistema externo real en Go demuestra que la salida del compilador tiene una aplicación práctica concreta más allá del ejercicio académico.

6. Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
- Levine, J. R. (2009). *flex & bison: Text Processing Tools*. O'Reilly Media.

7. Bibliografía

- GNU Project. (n.d.). *Bison - The Yacc-compatible Parser Generator*. Free Software Foundation.
- GNU Project. (n.d.). *Flex - The Fast Lexical Analyzer*. Free Software Foundation.
- The Go Programming Language. (n.d.). *Documentation*. Google. <https://go.dev/doc/>

- Docker Inc. (n.d.). *Docker Compose Documentation*. <https://docs.docker.com/compose/>